

Lab 4: Vibe Coding – Space Invaders

Debugging & Enhancement

- Repository: https://github.com/rishabnaveen/46_space-invaders
- Lab Goal: Prompt-driven bug fixing and feature addition in under 3–4 attempts
- Key Tasks: Refine Collision Detection (Task 1) & In-Game Game Over Screen (Task 2)

Prompt 1 (Initial Setup & Deliverables Roadmap)

User:

I have forked the repository https://github.com/SETAPESU26/46_space-invaders for Lab 4: Vibe Coding. Please review the lab requirements, analyze the deliverables from the `README.md`, and give me an exact step-by-step roadmap to complete the tasks.

AI Assistant:

Here is your step-by-step roadmap for Lab 4: VibeCoding:

1. Clone & Setup: Clone your forked repo and install dependencies (`pygame / pygame-ce`).
2. Baseline Recording: Run the initial game (`python main.py`) and record a 10-second video of the original broken state (bullets passing through enemies, abrupt exit on defeat). Save as `before.mp4`.
3. Task 1 (Collision Detection): Fix the collision bug where bullets pass straight through grouped enemies due to modifying the bullet list while iterating over it.
4. Task 2 (Game Over & Score Screen): Replace the terminal-only exit with an in-game graphical overlay displaying the final score, with options to restart (R) or quit (Q).
5. After Recording & Submission: Record a 10-second video of the working game (`after.mp4`), push changes to your personal GitHub fork, and compile deliverables into the `Lab-4/` folder.

Prompt 2 (Code Review & Bug Diagnosis)

User:

Here is my game engine code from `game/game_engine.py`. Please diagnose why bullets sometimes pass straight through enemies without registering a hit, and check the missing Game Over condition:

python

```
# Excerpt from game/game_engine.py
for bullet in self.player_bullets:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().collidect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            break

if self.game_over and not getattr(self, "_game_over_logged", False):
    print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

AI Assistant:

Root Cause Diagnosis:

1. Task 1 (Collision Bug): `self.player_bullets.remove(bullet)` mutates the `player_bullets` list while actively iterating over it. In Python, removing an item from a list during iteration shifts all subsequent elements left, causing the loop index to skip the next bullet. When multiple collisions occur on the same frame, hits fail to register.
2. Task 2 (Game Over Defect): When the player is hit or enemies reach the bottom, the game only logs `Game over! Final score:` to the console and freezes or terminates. There is no graphical GUI overlay showing the final score or offering restart/replay controls.

Prompt 3 (Bug Fixes & Feature Implementation)

User:

Please provide the complete refactored `game/game_engine.py` that resolves both Task 1 (safe collision detection) and Task 2 (graphical Game Over screen with score and restart/quit controls).

AI Assistant:

Here is the complete refactored `game/game_engine.py`:

python

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet
WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
GRAY = (200, 200, 200)
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.font = pygame.font.SysFont("Arial", 28)
        self.large_font = pygame.font.SysFont("Arial", 50, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 20)
        self.reset()
    def reset(self):
        """Resets the game state for a new session or replay."""
        self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(self.width)
        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        self.enemy_fire_chance = 0.01
        self.score = 0
        self.game_over = False
        self.game_won = False
    def handle_event(self, event):
        if self.game_over or self.game_won:
```

```

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                self.reset()
            elif event.key == pygame.K_q:
                pygame.event.post(pygame.event.Event(pygame.QUIT))
        return
    if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
        if self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y,
direction=-1))
            self._shoot_cooldown = 15
    def handle_input(self):
        if self.game_over or self.game_won:
            return
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)
    def update(self):
        if self.game_over or self.game_won:
            return
        if self._shoot_cooldown > 0:
            self._shoot_cooldown -= 1
        self.enemy_grid.move()
        if len(self.enemy_grid.alive_enemies()) == 0:
            self.game_won = True
            return
        for enemy in self.enemy_grid.alive_enemies():
            if random.random() < self.enemy_fire_chance:
                bullet_x = enemy.x + enemy.width // 2
                self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height,
direction=1))
        for bullet in self.player_bullets:
            bullet.move()
        for bullet in self.enemy_bullets:
            bullet.move()
        self.player_bullets = [b for b in self.player_bullets if not
b.off_screen(self.height)]
        self.enemy_bullets = [b for b in self.enemy_bullets if not
b.off_screen(self.height)]
        # --- Task 1 Fix: Safe Collision Detection ---
        bullets_to_remove = set()
        for bullet in self.player_bullets:
            bullet_rect = bullet.rect()
            for enemy in self.enemy_grid.alive_enemies():
                if enemy.alive and bullet_rect.colliderect(enemy.rect()):
                    enemy.alive = False
                    bullets_to_remove.add(bullet)
                    self.score += 1

```

```

        break
    self.player_bullets = [b for b in self.player_bullets if b not in
bullets_to_remove]
    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.game_over = True
            break
    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True
def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())
    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())
    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())
    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))
    # --- Task 2: Game Over / Victory GUI Screen ---
    if self.game_over or self.game_won:
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 190))
        screen.blit(overlay, (0, 0))
        title = self.large_font.render("GAME OVER" if self.game_over else "YOU
WIN!", True, RED if self.game_over else GREEN)
        score_summary = self.font.render(f"Final Score: {self.score}", True,
WHITE)
        controls = self.small_font.render("Press 'R' to Restart | 'Q' to
Quit", True, GRAY)
        screen.blit(title, title.get_rect(center=(self.width // 2, self.height
// 2 - 60)))
        screen.blit(score_summary, score_summary.get_rect(center=(self.width //
2, self.height // 2)))
        screen.blit(controls, controls.get_rect(center=(self.width // 2,
self.height // 2 + 50)))

```

Prompt 4 (Verification, Testing & Submission)

User:

I tested the game with the new code: hits register accurately, and the Game Over screen displays properly with final score and restart options. I recorded the after video and committed the changes using: `git commit -m "Fix collision detection and add graphical game over screen"` How do I complete my lab submission?

AI Assistant:

Great job! To finalize your submission:

1. Push your code: Run `git push origin main` to update your personal GitHub fork (do not open a PR to upstream SETAPESU26).
2. Assemble the Lab-4/ Folder:
 - before_video.mp4

- `after_video.mp4`
 - **Updated source code** (`main.py`, `game/game_engine.py`)
 - `chat_history.pdf` (This exported log)
3. Push the `Lab-4/` folder to your main coursework lab repository.